Final Project Report

CurioEd: a shared curriculum, a private record of learning, and an early-warning list a teacher can read.

https://github.com/aushakanuj/education-platform

1. Purpose

CurioEd is a school platform for three jobs that usually get split across different tools. An administrator publishes one curriculum for a grade and a subject. Each enrolled student studies that same curriculum through a private learning record, sits the same quiz, and sees their own result. A teacher or an administrator can open a short list of students who may need a conversation, and every name on that list carries the reason it appeared.

The repository is github.com/aushakanuj/education-platform. This report describes the system that is in that repository: what it does, how the early-warning rules were chosen, what the synthetic school is for, and what the system refuses to do. Paths below are from the repository root.

2. Constraints the work had to respect

On 19 August 2026 the group demonstrated the student, teacher, and administrator paths to mentor Peeyush. The minutes of that review (docs/mom/week2.md) record two directions. Analytics had to become a method someone can justify: which signals, which cut-offs, and why. AI help had to stay inside material the school had already approved, with a person in the loop, rather than inventing lesson facts on the fly. Sponsor feedback, relayed by Pranit from Lakshmi, asked for personal feedback after a quiz. Peeyush tied that feedback to question-level tags, so a weak area can be named instead of handing the student a single percentage.

The sponsor did not provide historical pupil records. The generator module says this directly: CBT is providing no data, so the platform builds its own school. The early-warning design note draws the consequence. A learned model would need two or three cohorts of real outcomes the sponsor could not confirm. The signed direction was a rule a person can read, not a score a model emits. The design note also forbids presenting these rules as a prediction about a real child. Every figure in the demo comes from a generated school.

3. Product model

The product is two directories over one curriculum tree. The tree is Academic Period, Grade, Subject, Topic, Subtopic.

SourceCurriculum belongs to the administrator. A subtopic holds published lesson versions and one common mastery quiz. Learner activity never overwrites that source.

StudentLearningDirectory belongs to one student. It points at the published lesson and quiz. It stores that student's progress and attempts. It does not keep a second copy of the file. A student reaches it only with an active account, an active period, a grade enrollment, and a grade–subject enrollment, and only when the source is published.

Teachers see evidence for classes they are assigned to teach. Administrators can read the institution directory without holding a student enrollment. A second institution sees nothing of the first. Roles in the schema are administrator, teacher, and student. Access is a JWT plus those roles and enrollments. The platform does not add a side door around them.

4. What is running

The proof-of-concept loop is: publish a lesson, study it, sit the common quiz, score it, and give staff an explainable flag. The pieces below are in the codebase and covered by the test suite. Items still marked as a design contract, rather than a shipped behaviour, are listed in section 9.

Area	Where it lives	State
Sign-in, refresh tokens, roles	modules/auth	Running. Demo school password is the fixture demo1234.
Periods, grades, subjects, enrollments, learning directory	modules/academics	Running. Students need active grade and grade–subject enrollments.
Lessons, progress, curriculum seed, PDF ingest	modules/materials, workers	Running. Approved markdown in docs/curriculum is the seed. PDF blobs stay on local disk.
Common quizzes and attempts	modules/assessments	Running. Default pass mark is 70%. Student APIs do not return answer keys.
student_360 register	modules/insights	Running. One read path for mastery, quizzes taken, and attendance.
Early-warning flags	modules/at_risk, teacher and admin /at-risk pages	Running. Staff only. Each flag names its drivers. Dismissal and viewing are audited.
Ask-the-data over the same flags	modules/text_to_sql, at-risk agreement tests	Running. A teacher question and the flag list are tested to return the same students.
Policy assistant	modules/assistant	Running for administrators. Pipeline is inject, validate, retrieve, summarize, over indexed documents.
Vector index	modules/rag, Postgres pgvector	Running. Ingest jobs are claimed with FOR UPDATE SKIP LOCKED.

Demo accounts for the synthetic Al Noor school, all with password demo1234: administrator fatima.almansouri@alnoor.school, teacher meera.krishnan@alnoor.school, student student1@alnoor.school (Adnan, Grade 6), and student97@alnoor.school (Aisha Rahman, Grade 8).

5. Early-warning method

5.1 What a flag is

A flag is a staff prompt to look at one student. It is not a grade, not a letter home, and not a label shown to the student. The design note docs/design/08-at-risk-early-warning.md separates it from the per-subtopic struggle signal a student may see on their own feedback page. Struggle says “this topic is weak.” At-risk says “a teacher should look at this child,” and it has to say why.

Every input is read from student_360. Subject mastery and quiz counts are per student per subject offering. Attendance is per student for the whole period, because that is the grain the register actually stores. A rule that pretended attendance was per subject would be wrong on this schema. The engine is a pure function in modules/at_risk/engine.py. It does not query the database. The service loads signals and stores the flags the function returns.

5.2 Rules

Mastery and attendance are judged separately. They are never averaged into one blend. A blend would hide “fine everywhere except mathematics” inside a middling composite, which is the case the design note exists to catch.

- Mastery level. Flag the subject when mastery_percent is below 60.
- Mastery trend. When the student has at least two attempts before the latest three, compare the mean of those earlier attempts with the mean of the latest three. Flag when the later mean is more than 15 points lower.
- Attendance level. Flag when attendance_percent is below 80. There is no attendance trend in this version: the register stores a cumulative percentage, not a series.
- A subject flag never lists attendance as one of its drivers. An attendance flag names no subject.
- A flag with an empty driver list cannot be constructed. That check is in the type itself (requirement AR-1).

Tiers follow from how many drivers fired, and from which kind:

Tier	When it is used
urgent	Two or more drivers on the same flag. In practice, low mastery and a sharp decline together.
attention	The only driver is the trend. The latest level can still sit at or above 60.
monitor	A single level driver: mastery below 60, or attendance below 80.

The cuts are closed on the safe side of the line. Mastery of exactly 60, a decline of exactly 15 points, and attendance of exactly 80 do not flag. Missing attendance produces no attendance flag and does not raise an error. Too little history to fill the trend window produces no trend driver. The level rule still applies.

5.3 Why these numbers

The design note refuses to invent a cut-off with no distribution behind it. The engine file records the calibration against the generated school:

- Mean mastery on the synthetic school is recorded as 71.3%, standard deviation 8.5. One and a half standard deviations below that mean is about 58.5%. The level cut is 60, just above that. The same note records that this flags about 11.5% of student–subject rows.
- A decline of more than 15 points is recorded for about 2.9% of groups that have enough history. It is meant to be uncommon. A slide of a few points is ordinary noise.
- Attendance at 80 sits in a gap the same note describes: a cluster near 70%, then the next mass of students at 86% and above. The old on-screen placeholder of 75 was a demo constant and has been removed from the roster.

These constants belong to this generated school. Reusing them on another school, without looking at that school's own distribution, would repeat the mistake the design note was written to stop. A different rule, exam eligibility, still uses 75% attendance (a student below that line cannot sit the end-of-term exam). That rule is not the early-warning cut. The two numbers are supposed to differ, and the code names them differently so they do not drift back into one shared constant.

5.4 A worked case: Aisha Rahman

Aisha Rahman is student S-00097 in Grade 8. She is planted by the generator so the engine has a case it must get right. The engine test records her mathematics sequence as it was read from the generated database on 29 August 2026. Chronological scores are 74, 65, 56, 47, 38. Her subject mastery is 56%. The design note records her other subjects between 61% and 77%, and her attendance at 62%.

Mathematics, level. 56 is below 60, so the mastery_percent driver fires. On its own that would be a monitor flag.

Mathematics, trend. The engine reads scores most-recent first: 38, 47, 56, 65, 74. The latest three average (38 + 47 + 56) / 3 = 47. The earlier two average (65 + 74) / 2 = 69.5. The decline is 69.5 − 47 = 22.5 points, which is past 15. The mastery_trend driver fires, and the test locks that 22.5.

Two drivers on the mathematics flag make the tier urgent. The flag names the mathematics offering. A strong subject for the same student produces no flag. The test for that pair asserts a single flag, and that its offering is mathematics.

Attendance. 62 is below 80, so a second flag fires with only an attendance_percent driver, tier monitor, and no subject id. Attendance is not copied onto the mathematics flag. The inverse is also tested: poor attendance does not leak into a subject driver. Because the attendance flag names no subject, the visibility rule sends it to administrators. The mathematics flag is visible to the teacher assigned to that offering, and to administrators.

5.5 The other planted shapes

- Attendance without weak marks. Every 17th student is generated at a 70% attendance rate with marks that stay clear of the mastery cut. The engine returns one flag, attendance only, tier monitor. A fine subject contributes nothing.
- Decline while the level still looks acceptable. A trend-only flag is tier attention. A level-only rule would have missed this student. That is why both conditions exist.
- A section gap. Two sections of the same subject are built so that one topic really does differ. The individual rule still emits ordinary per-student flags. It does not invent a cohort object that would hide a teaching or content gap inside a pile of identical personal flags. The design note leaves the cohort story itself out of scope, and requires the engine not to paper over it.

5.6 What staff are allowed to do with a flag

The design note binds five behaviours. The API tests exercise them.

Id	Rule
AR-1	No flag without at least one named driver (metric, value, comparison, window).
AR-2	A flag never sends a message to a student, parent, or guardian.
AR-3	A flag never writes a grade or any other academic record.
AR-4	Dismissal is optional-note, mandatory-audit: who, when, which flag.
AR-5	Opening the flag list is an audited read, scoped like any other student_360 read.

Students receive 403 on the flag list, on dismiss, and on recompute. A teacher sees flags only for subjects they teach. An administrator sees more, including attendance-only flags, which a subject teacher does not see. An administrator of another institution sees an empty list. Dismissing a flag

outside one's scope returns 404, the same shape as a missing flag, so the response does not reveal that the row exists. After dismissal the flag leaves the active list, and both the view and the dismissal write an audit event.

6. What the tests fix in place

Unit tests in backend/tests/test_at_risk_engine.py cover the boundary values, the empty-driver rejection, Aisha's mathematics sequence, the attendance-only shape, and the refusal to merge two weak subjects into one composite. API tests in backend/tests/test_at_risk_api.py cover role, tenant, dismiss, and audit.

A separate agreement fixture, not the 240-student school, checks that the staff question-answering path and the flag endpoint name the same students. In that fixture the teacher list has 9 students, all in mathematics: eight planted low-mastery students and one trend-only student. A legacy query that also counted low attendance returned 12. The four attendance-only students are on the administrator list and absent from the teacher list. The test exists so a later prompt cannot quietly resurrect the legacy count.

Continuous integration on pull requests runs Ruff format, Ruff lint, mypy on the backend, Alembic upgrade, and pytest with an 80% coverage gate, and the frontend Vitest suite in parallel. Live calls to a paid language model are opt-in and are not part of that job.

7. Data

The one-page Data Note in this package is the data statement. In short: there is no extract of real pupils. The default school is Al Noor International, Grades 6 to 10, two sections per grade, 24 students per section (240 students), five subjects, five quizzes per subject, ten term weeks, seed 20260812. The same command rebuilds the same school. Names, marks, and attendance are authored so that the planted cases in section 5 have a determinate answer. Nothing in that school is evidence about a living child.

8. Grounded assistance

The administrator policy assistant is a fixed pipeline: check the question for injection, validate it, retrieve chunks from the school's indexed documents, then summarize. Retrieval is pgvector over those chunks. If no OpenRouter key is configured, a heuristic injection check and a stub summary still run, so the path is exercisable without a paid model. The key is an environment variable. It is not part of this submission.

Student-facing answers are a narrower contract. Quiz APIs do not return the answer key. Scoring happens on the server. The design for lesson generation is the same idea the mentor asked for in August: a draft has to come from approved sources, and a person publishes it. A student grounded tutor, sitting on top of the same index, is still a later piece of work. The policy assistant is the part that runs today.

9. Limits

- The school is synthetic. Thresholds tuned on it show that the mechanism distinguishes the planted cases. They do not show that the mechanism predicts real outcomes.
- The engine is rules, by decision. A learned scorer is out of scope, not a backlog item waiting on more of the same synthetic rows.

- Four-pillar evaluation snapshots (marks, anonymised peer band, weak outcomes, trend) are specified in docs/design/06-analytics-and-comparison-insights.md. Peer bands require at least five scored students and 80% completion before a comparison is shown. Stored snapshots of that full object are not the shipped record. What ships today is attempt scoring, feedback from those attempts, student_360, and the early-warning flags.
- Some teacher screens outside the at-risk list are still fed by fixtures. The at-risk pages call the live API.
- There is no parent role, and a flag cannot notify a family.
- Lesson files for the proof of concept live on the application disk. Shared object storage is not in this version.
- A native mobile client is a target architecture, not a shipped client. The web client is the one that runs.

10. Reproducing the analysis

1. Start Postgres with pgvector from the repository root: docker compose up -d postgres.
2. From backend/, apply migrations: uv run alembic upgrade head.
3. Rebuild the school: uv run python -m education_platform.modules.synthetic.cli. The default seed is 20260812.
4. The engine does not need the database. From backend/: uv run pytest tests/test_at_risk_engine.py tests/test_at_risk_api.py tests/test_at_risk_assistant_agreement.py.
5. Accounts, curls, and the web client are in the repository README.

The Code and Analysis Repository note in this package is the short map of those paths and the link to submit for that line of the LMS package.

11. People

Mentor for the 19 August 2026 review: Peeyush. Sponsor contact named in those minutes: Lakshmi. The minutes also list tanmay among the attendees. This repository has no commit under that name and no surname for that person, so tanmay is not on the signature block. Add a row on the declaration if tanmay is a submitting member.

Human commit counts below exclude automated-agent commits. They are a trace of who touched the tree, not a split of ownership. Everyone listed signs one group declaration.

Name	Trace in the repository
Aushak Anuj Dendukuri	Repository owner (github.com/aushakanuj). Largest set of human commits on the platform.
Pranit Daggupati	Platform commits. Minutes record him carrying sponsor feedback into the 19 August review.
Suparna Dhumale	Platform commits. Minutes credit the Student 360 view.
Arathe Sankar	Struggle-flag readout and the student feedback presentation (TEAM-READOUT.md, September 2026).
Rahul Vamshikanth Gunda	Commits on the repository under Rahul G.

12. This package

The LMS notice asks for one submission per group, due 25 October 2026 at 11:55 PM. The notice does not name a time zone. The group's recorded meeting used India Standard Time. The package beside this report is:

- This report, as PDF and as DOCX.
- Assignment Submission Form.
- Data Note, one page.
- Originality and Confidentiality Declaration, with signature lines left blank for the group.
- Plain-Language Summary.
- Code and Analysis Repository note, pointing at the GitHub repository.
- A pre-upload checklist. The notice points at Section 9 of an Instructions and Best Practices guide. That guide is not in the repository, so the checklist covers the items the notice itself named, and it says so.

The narrative documents were drafted with Cursor from the repository's code, tests, and design notes. The declaration states that. Signing means the group has checked the statements against the code.